CHAPTER 1

INTRODUCTION

1.1 Project Synopsis

The Alumni Management System is a web-based application designed to strengthen the relationship between educational institutions and their alumni by providing a centralized and secure digital platform. The system addresses the challenge of maintaining long-term communication with alumni after graduation by offering an organized way to store alumni records, manage events, share career opportunities, and facilitate interaction.

The application allows alumni to register, manage their profiles, view institutional updates, apply for job opportunities, and participate in events. Administrators have complete control over the system, including verifying alumni accounts, approving job postings, managing events and news, and monitoring overall platform activity. Role-based access control ensures that only authorized users can access sensitive features.

The system is developed using React.js for the frontend, Node.js with Express.js for the backend, and MySQL as the database. Secure authentication is implemented using JSON Web Tokens (JWT), and passwords are protected using hashing techniques. The use of RESTful APIs ensures smooth communication between the frontend and backend components.

Overall, the Alumni Management System provides an efficient, scalable, and secure solution for alumni engagement, enabling institutions to foster professional networking, mentorship, and continuous involvement of alumni in institutional growth.

Project Overview

The Alumni Management System is a web-based application developed to establish a strong and continuous connection between an educational institution and its alumni. The project aims to provide a centralized platform for managing alumni information, facilitating communication, and promoting alumni participation in institutional activities.

The system enables alumni to register, maintain their profiles, view events and announcements, apply for job opportunities, and interact with the administration. Administrators are responsible for verifying alumni accounts, managing events, approving job postings, publishing news, and monitoring system activities.

Built using React.js, Node.js with Express.js, and MySQL, the application follows a secure and scalable architecture. Features such as role-based access control, JWT-based authentication, and structured database design ensure data security, reliability, and efficient system management.

Key Features

1. User Registration and Authentication

- Secure alumni registration and login using JWT (JSON Web Token) authentication.
- Role-based access control with Administrator and Alumni roles.
- Alumni profile management including personal, academic, and professional details.
- Password recovery and account verification features.

2. Alumni Directory and Networking

- Centralized alumni database with advanced search and filter options (Department, Batch, District).
- Profile visibility controls to ensure user privacy.
- Direct interaction and networking among alumni members.

3. Job and Career Portal

- Alumni-driven job posting and job opportunity sharing.
- Admin approval workflow for job postings.
- Easy browsing and application access for alumni users.

4. Events, Chapters, and Activities

- Management of alumni events, reunions, and institutional programs.
- Support for regional and batch-wise alumni chapters.
- Event notifications and participation tracking.

5. Communication and Notifications

- Real-time push notifications for announcements and updates.
- News ticker for college and alumni association news.
- Batch messaging for targeted communication.

6. Media Gallery

- Image slider, photo gallery, and video pop-up features.
- Showcasing alumni events, campus memories, and achievements.

7. Secure Administration Panel

- Admin dashboard for managing users, jobs, events, media, and notifications.
- Content moderation and system configuration controls.
- Secure access to administrative functionalities.

Technological Stack

- Frontend: React.js (for dynamic UI components and responsive design)
- Backend: Node.js with Express.js (for RESTful APIs and server-side logic)
- Database: MySQL / PostgreSQL using Sequelize ORM
- Authentication: JWT-based secure authentication system
- Deployment: Cloud-based platforms such as AWS, Vercel, or Heroku

1.2 Organization Profile

Introduction:

Arul Anandar College ERP in college dedicated to providing innovative software solutions and services to educational institutions. With a focus on empowering students and organizations through technology, Arul Anandar College ERP has established itself as a trusted partner in the digital transformation journey.

Vision:

Our vision at Arul Anandar College is to revolutionize the way educational institutions and business operate by leveraging cutting-edge. We aspire to empower students, educators, and professionals alike, fostering a culture of innovations, collaboration, and continuous learning.

Mission:

Arul Anandar College ERP, our mission is to deliver exceptional software services that exceed Student expectations, drive efficiency, and enable growth. We are committed to nurturing talent, fostering creativity, and providing unparalleled support to our clients, ensuring their success in an ever-evolving digital landscape.

Core Values:

Innovations: We embrace innovations as a catalyst for transformation, constantly pushing boundaries and challenging the status quo to deliver groundbreaking solutions.

Quality:

We are committed to excellence in everything we do, adhering to the highest standards of quality, reliability and performance.

Student Centricity:

Our students are at the heart of everything we do. We listen attentively to their needs, anticipate their requirements and strive to exceed their expectations.

Integrity:

We conduct ourselves with honesty, transparency and integrity in all our interactions, building trust and fostering long term relationships with our clients, partners and stakeholders.

Collaboration:

We believe in the power of collaboration and teamwork, leveraging diverse perspectives and expertise to achieve shared goals and drive collective success.

CHAPTER 2

PROJECT OBJECTIVES

2.1 Problem Definition

Traditional alumni management system in many institutions rely on manual records, social media groups, or disconnected communication channels. These approaches are inefficient, difficult to maintain, and fail to provide a centralized platform for alumni engagement. Managing alumni data manually is time-consuming, prone to errors, and lacks real-time updates. Communication between the institution and alumni is often irregular, leading to low participation in events, limited networking opportunities, and poor dissemination of job-related information.

Additionally, the absence of a dedicated digital platform makes it difficult to track alumni achievements, manage events, and support professional collaboration. This project addresses these challenges by providing a centralized, secure, and scalable Alumni Management System that enhances connectivity, engagement, and long-term relationships between alumni and the institution.

Problems Identified:

- Difficulty in managing alumni data manually: Maintaining large volumes of alumni records using traditional methods is inefficient and error-prone.
- Lack of centralized communication: Important announcements, events, and updates often fail to reach all alumni effectively.
- Limited networking and career support: Alumni lack a structured platform to connect professionally and share job opportunities.
- Poor event and chapter management: Organizing reunions, regional chapters, and alumni activities without a centralized system is challenging.
- Inefficient data tracking and reporting: Manual systems lack proper analytics, historical data management, and reporting capabilities.

2.2 Scope of the Project Proposal

The proposed Alumni Management System aims to overcome the identified challenges by providing a structured, feature-rich, and scalable web-based platform for alumni engagement and management. The system includes the following modules and features:

1. User Management

- Multiple user roles: Administrator and Alumni.
- Role-based access control (RBAC) to ensure secure operations.
- Alumni profile management including personal, academic, and professional details.

2. Alumni Directory and Networking

- Centralized alumni database with advanced search and filtering options.
- Secure profile visibility controls to protect user privacy.
- Platform for alumni interaction and professional networking.

3. Job and Career Management

- Alumni-driven job posting and opportunity sharing.
- Admin verification and approval of job postings.
- Easy access to job listings for alumni members.

4. Events, Chapters, and Activities

- Creation and management of alumni events, reunions, and college programs.
- Support for regional, departmental, and batch-wise chapters.
- Event notifications and participation tracking.

5. Communication and Notifications

- Push notifications for announcements and important updates.
- News ticker for college and alumni association news.
- Batch messaging for targeted communication.

6. Media and Content Management

- Image gallery, video pop-ups, and image sliders for showcasing alumni events.
- Storage and management of photos and media content.

7. Security and Data Integrity

- Secure authentication using JWT-based login.
- Admin-controlled content moderation.
- Data validation and protection against unauthorized access.

8. Scalability and Accessibility

- Scalable architecture capable of handling a growing alumni base.
- Cloud-based deployment for high availability.
- Responsive design for access on desktops, laptops, tablets, and mobile devices.
- Future scope for multi-language support and enhanced accessibility features.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Demerits of the Existing System

The existing alumni management practices in many institutions rely on manual records, spreadsheets, social media groups, or disconnected communication platforms. These approaches suffer from several limitations that affect efficiency, accuracy, and alumni engagement.

Manual Process

Alumni details are often collected and maintained manually through paper records or basic digital files. Updating alumni information such as contact details, employment status, or achievements requires significant effort and is not performed regularly.

Human Errors

Manual data entry increases the risk of errors such as incorrect records, duplicate entries, and outdated information. These inaccuracies reduce the reliability of alumni data.

Limited Communication

Communication between the institution and alumni is irregular and unstructured. Important announcements, events, and job opportunities may not reach all alumni effectively.

Poor Networking and Career Support

Without a centralized platform, alumni find it difficult to connect with peers, share professional opportunities, or collaborate for career growth.

Data Management Challenges

Storing and retrieving alumni records is inefficient. Historical data is difficult to track, and records may be lost or become inaccessible over time.

Security Risks

Existing systems lack proper authentication and authorization mechanisms, making alumni data vulnerable to unauthorized access and misuse.

3.2 Newly Proposed System

The Alumni Management System overcomes the limitations of the existing system by providing a centralized, secure, and scalable web-based platform.

Centralized Alumni Management

All alumni information is stored in a structured digital database, enabling easy updates, search, and retrieval of records.

Secure and Role-Based Access

The system uses secure authentication and role-based access control to differentiate between administrators and alumni users.

Enhanced Communication

The portal provides push notifications, news updates, and batch messaging to ensure effective and timely communication.

Networking and Career Support

Alumni can connect with each other, view profiles, and share job opportunities through a dedicated job portal.

Event and Chapter Management

The system supports alumni events, reunions, and regional or batch-wise chapters with proper tracking and notifications.

Scalable Architecture

Built using modern web technologies, the system supports a growing alumni base without performance degradation.

3.3 Challenges of the Proposed System

Despite its advantages, the Alumni Management System faces certain challenges:

Ensuring Data Security

Protecting sensitive alumni data from unauthorized access and cyber threats is critical and requires strong authentication and encryption mechanisms.

Handling High User Traffic

During major events or announcements, the system must efficiently handle increased user activity without performance issues.

User Adoption

Encouraging alumni from different age groups and technical backgrounds to actively use the system can be challenging.

System Maintenance

Regular updates, backups, and technical maintenance are required to ensure smooth operation.

Integration Challenges

Seamless integration between frontend, backend, and database components must be maintained for optimal performance.

3.4 Feasibility Analysis

The feasibility of implementing the Alumni Management System is analysed as follows:

Technical Feasibility

- Modern web technologies such as React.js and Node.js provide a reliable and scalable framework.
- Cloud-based deployment ensures availability, scalability, and data backup.

Operational Feasibility

- The system simplifies alumni management and communication processes.
- Administrators can manage users, events, and content efficiently with minimal training.

Economic Feasibility

- Reduces costs related to manual record maintenance and communication.
- Long-term benefits outweigh initial development and deployment costs.

3.5 Use Case Analysis

The Alumni Management System supports the following primary use cases:

1. User Registration and Authentication

- Alumni register and log in using secure credentials.
- Role-based access ensures secure usage.

2. Alumni Profile Management

- Alumni can create and update personal, academic, and professional details.
- Administrators can verify and manage profiles.

3. Alumni Directory and Networking

- Users can search and connect with alumni based on department, batch, or location.

4. Job and Career Management

- Alumni can post and view job opportunities.
- Admin approval ensures authenticity of postings.

5. Event and Chapter Management

- Administrators create events and chapters.
- Alumni receive notifications and participate in activities.

6. Communication and Reporting

- Admins send announcements and notifications.
- System generates reports on alumni participation and engagement.

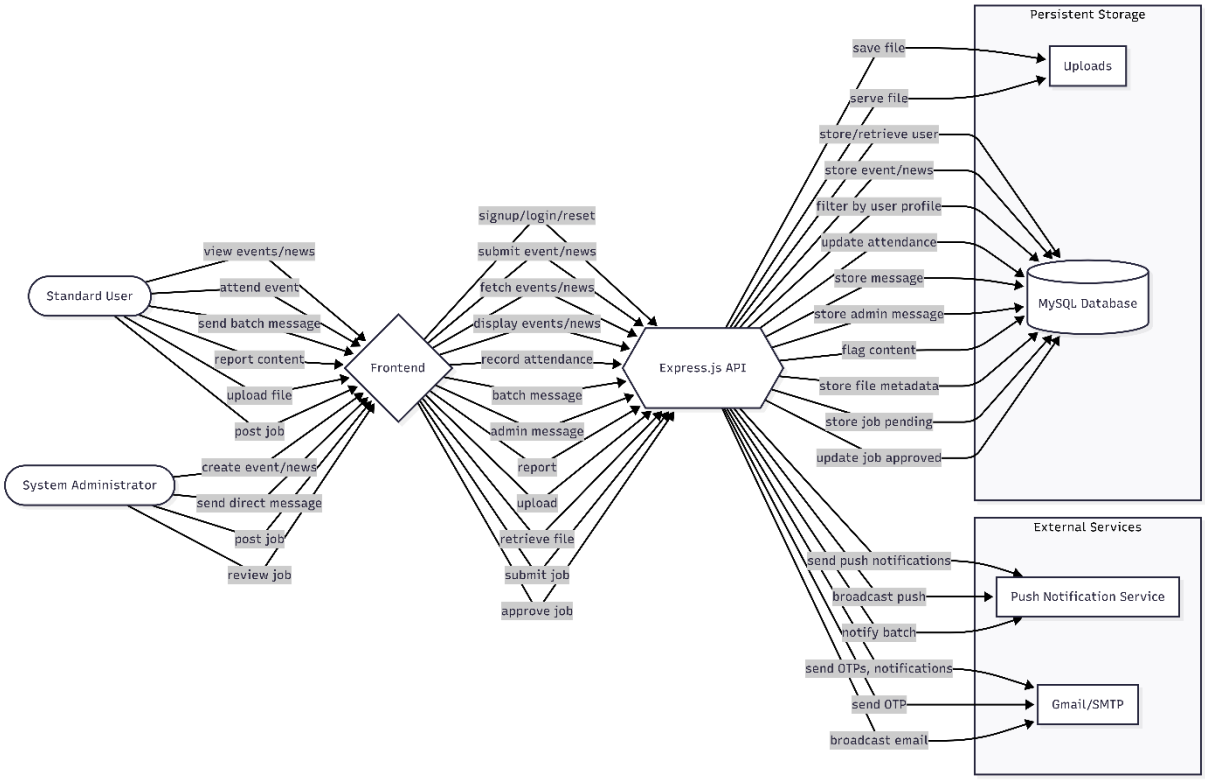


Figure: Data Flow Diagram of Alumni Management System

CHAPTER 4

SYSTEM DESIGN

4.1 Configuration Specification

To ensure the smooth operation of the **Alumni Association Portal**, the following configuration requirements are necessary.

Hardware Requirements

The system is designed to operate efficiently in both local and cloud environments while supporting multiple concurrent users.

Server Requirements

- Cloud-based Hosting: AWS, Google Cloud, or Vercel for scalability, high availability, and performance.
- Dedicated Server (Optional):
    - Minimum **8 GB RAM** for handling concurrent requests.
    - Multi-core processor (Intel Core i5 / AMD Ryzen 5 or higher).
    - **SSD storage (256 GB or higher)** for fast data access and backups.
    - High-speed internet connectivity to minimize latency.
- Backup & Disaster Recovery:
    - Automated scheduled backups using cloud storage.
    - Redundant servers and failover mechanisms for business continuity.

Client Requirements

- Any modern device (Desktop, Laptop, Tablet, Smartphone) with internet access.
- Supported Browsers:
    - Google Chrome (recommended)
    - Mozilla Firefox
    - Microsoft Edge
- Operating System Compatibility:
    - Windows 10+, macOS, Linux
    - Android and iOS devices with updated browsers

4.2 Modularity Specification

The system follows a modular architecture to improve maintainability, scalability, and flexibility.

User Management Module

- Handles alumni registration, authentication, and role-based access control (Admin, Alumni).
- Allows users to update profiles and manage passwords.

Alumni Directory Module

- Manages alumni profiles with academic, personal, and professional information.
- Provides advanced search and filtering options.

Job & Career Module

- Enables alumni to post and view job opportunities.
- Admin verification and approval of job postings.

Events & Chapters Module

- Manages alumni events, reunions, and regional or batch-wise chapters.
- Tracks participation and sends notifications.

Media Gallery Module

- Manages photos, videos, and image sliders.
- Showcases alumni activities and institutional events.

Communication Module

- Handles push notifications, announcements, and batch messaging.

Security Module

- Implements secure JWT-based authentication.
- Ensures data privacy and role-based access control.

4.3 Data Design Specification

The system uses a relational database managed through **Sequelize ORM**, designed to store and manage alumni data efficiently.

Database Schema

Users Table

- Stores alumni and admin details.
- Example Fields: user_id, name, email, role, department, batch, district, created_at.

Jobs Table

- Stores job postings shared by alumni.
- **Example Fields:** job_id, title, company, location, description, posted_by, status, created_at.

Events Table

- Stores alumni events and reunion details.
- Example Fields: event_id, title, date, venue, description, created_by.

Chapters Table

- Stores regional and batch-wise chapter details.
- Example Fields: chapter_id, name, region, coordinator_id.

Media Table

- Stores gallery images and videos.
- Example Fields: media_id, type, url, uploaded_by, uploaded_at.

Notifications Table

- Stores announcements and messages.
- Example Fields: notification_id, title, message, target_group, created_at.

4.4 Form Design Specification

Forms are designed to ensure usability, accessibility, and smooth interaction.

User Registration Form

- Fields: Name, Email, Department, Batch, District, Password.
- Features: Input validation, email verification.

Login Form

- Fields: Email, Password.
- Features: Secure authentication and session handling.

Alumni Profile Form

- Fields: Personal, academic, and professional details.
- Features: Profile edit, privacy controls.

Job Posting Form

- Fields: Job title, company name, location, description.
- Features: Admin approval workflow.

Event Creation Form

- Fields: Event title, date, venue, description.
- Features: Scheduling and notifications.

Feedback Form

- Fields: Feedback rating, comments, suggestions.
- Features: Anonymous submission option.

4.5 Report Design Specification

Reporting features support analysis and decision-making for administrators.

Alumni Engagement Reports

- Tracks alumni registrations, active users, and participation levels.
- Displays trends over time.

Event Participation Reports

- Shows attendance details for events and reunions.
- Identifies highly engaged chapters.

Job Portal Reports

- Summarizes job postings, approvals, and applications.

Statistical Reports

- Visual representation using bar charts, line graphs, and pie charts.
- Filters based on batch, department, and date range.

Export Options

- Reports can be downloaded in PDF format.
- Enables secure sharing with institutional authorities.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Implementation Tool Specification

The development and deployment of the Alumni Management System require a reliable and modern set of tools to ensure scalability, security, and an enhanced user experience.

Frontend Development

The frontend is developed using modern web technologies to provide a responsive and interactive interface for alumni and administrators.

- React.js: JavaScript library for building dynamic and component-based user interfaces.
- Redux / Context API: Manages global application state efficiently.
- Tailwind CSS: Ensures a responsive, mobile-friendly, and consistent design.
- Axios: Handles API requests for seamless data communication with the backend.
- Framer Motion: Adds smooth animations and enhances user experience.

Backend Development

The backend manages core functionalities such as authentication, alumni data management, job postings, events, and notifications.

- Node.js: Server-side JavaScript runtime for efficient backend processing.
- Express.js: Lightweight framework for building RESTful APIs.
- JWT (JSON Web Tokens): Provides secure authentication and session management.
- bcrypt: Encrypts passwords to enhance data security.

Database Management

A relational database is used to efficiently manage structured alumni data.

- MySQL / PostgreSQL: Stores alumni profiles, job postings, events, and notifications.
- Sequelize ORM: Simplifies database interactions and enforces data consistency.

Development Tools

- Visual Studio Code (VS Code): Primary development environment.
- Postman: API testing and debugging tool.

- Git & GitHub: Version control and collaborative development.
- Jest: Testing framework to ensure application reliability.

Deployment Tools

- Vercel / Netlify: Hosts the frontend application.
- AWS: Deploys the backend services and database.
- CI/CD Pipelines: Automates testing and deployment for faster and reliable updates.

5.2 Code Metrics Specification

To ensure high-quality and maintainable code, the following metrics are applied:

Code Quality

- ESLint: Enforces coding standards and best practices.
- Prettier: Maintains code readability and formatting consistency.

Code Complexity

- Low Cyclomatic Complexity: Simplifies debugging and maintenance.
- Modular Design: Promotes separation of concerns and scalability.

Code Reusability

- Component-Based UI: Reusable React components minimize redundancy.
- Modular API Architecture: Separate routes, controllers, and services for flexibility.

Code Efficiency

- Optimized API Endpoints: Reduce server response time.
- Database Indexing: Ensures fast retrieval of alumni-related data.
- Caching Mechanisms**:** Improves performance for frequently accessed data.

5.3 Technology Specification

The Alumni Management System is built using the following technologies:

Frontend Technologies

- React.js: Responsive and interactive user interface.
- Redux / Context API: State management across components.
- CSS Frameworks: Bootstrap and Tailwind CSS for styling.

Backend Technologies

- Node.js & Express.js: Backend logic and API handling.
- JWT & bcrypt: Secure authentication and data protection.
- Multer: Handles file uploads for profile images and media gallery.

Database Technologies

- MySQL / PostgreSQL: Structured and scalable data storage.
- Sequelize ORM: Database schema management and querying.

API & Integration

- RESTful APIs: Enable seamless communication between frontend and backend.
- WebSockets (Optional): Supports real-time notifications and updates.
- Third-Party Integrations: Email services and cloud storage (if required).

Deployment Technologies

- Vercel / Netlify: Frontend hosting.
- AWS: Backend and database deployment.
- CI/CD Pipelines: Continuous integration and deployment for system stability.

CHAPTER 6

SYSTEM TESTING

6.1 Test Case Specification

Comprehensive test cases are defined to validate each functional module of the Alumni Association Portal. The major test cases are outlined below:

User Registration

- Objective: To ensure alumni can register successfully with valid information.
- Process: Enter valid details such as name, email, department, batch, district, and password, then submit the form.
- Expected Result: The system confirms successful registration and redirects the user to the login page.
- Negative Scenario: Registering with an already existing email should display an error message such as *"Email already exists."*

User Login

- Objective: To verify secure login functionality.
- Process: Enter valid email and password credentials.
- Expected Result: User is redirected to the respective dashboard (Admin or Alumni).
- Negative Scenario: Invalid credentials should display *"Invalid email or password."*

Alumni Profile Management

- Objective: To ensure alumni can update profile details.
- Process: Modify personal or professional details and save changes.
- Expected Result: Updated details are saved successfully in the database.
- Negative Scenario: Leaving mandatory fields empty should trigger validation errors.

Job Posting

- Objective: To verify alumni can submit job postings.
- Process: Enter job details and submit for approval.
- Expected Result: Job is saved and marked as *"Pending Approval."*

- Negative Scenario**:** Missing mandatory job details should show validation warnings.

Event Creation

- Objective: To ensure administrators can create alumni events.
- Process: Enter event details such as title, date, venue, and description.
- Expected Result: Event is successfully created and visible to alumni.
- Negative Scenario: Invalid dates or missing fields should display error messages.

Notification Delivery

- Objective: To validate announcement and notification delivery.
- Process: Admin sends notifications to selected alumni groups.
- Expected Result: Alumni receive notifications instantly on their dashboard.

6.2 Test Data Specification

Test data is designed to validate normal, boundary, and invalid input scenarios.

User Data

Valid Data:

- Name: Ramesh Kumar
- Email: ramesh.kumar@example.com
- Department: Computer Science
- Batch: 2018
- Password: Alumni@123

Invalid Data:

- Name: "" (empty)
- Email: ramesh@com (invalid format)
- Password: "12345" (less than required length)

Job Posting Data

Valid Data:

- Job Title: Software Developer
- Company: ABC Technologies

- Location: Chennai

Invalid Data:

- Job Title: "" (empty field)
- Company: ""

Event Data

Valid Data:

- Event Title: Alumni Reunion 2025
- Date: 15-06-2025
- Venue: College Auditorium

Invalid Data:

- Event Date: Past date
- Venue: "" (empty)

6.3 Data Validation Specification

Data validation ensures correctness, consistency, and security of user inputs.

User Registration Validation

- Name: Must not be empty and contain only alphabets.
- Email: Must follow standard email format.
- Password:
    - Minimum 8 characters.
    - Must include uppercase, lowercase, numeric, and special characters.
- Department & Batch: Mandatory fields.

Profile Update Validation

- Ensure valid contact details and professional information.
- Prevent invalid or duplicate data entries.

Job Posting Validation

- Job title, company, and description must not be empty.
- Admin approval required before publishing.

Event Creation Validation

- Event date must be a future date.
- Venue and description are mandatory.

Feedback Validation

- Feedback comments limited to 500 characters.
- Ratings restricted between 1 and 5.

6.4 Test Process Specification

The testing process follows a structured approach to ensure system reliability and quality.

1. Unit Testing

- Tests individual components such as forms, API endpoints, and database operations.
- Tools Used: Jest, Postman.

2. Integration Testing

- Ensures proper interaction between frontend, backend, and database.
- Example scenarios include login, job posting, and event creation.

3. System Testing

- Validates end-to-end functionality of the portal.
- Example: Alumni registration → profile update → job posting → admin approval.

4. User Acceptance Testing (UAT)

- Conducted with actual alumni and administrators.
- Feedback is collected to improve usability and performance.

5. Regression Testing

- Ensures new features do not impact existing functionalities.
- Performed after every system update.

6. Performance Testing

- Evaluates system behavior under multiple concurrent users.
- Ensures smooth performance during peak alumni activity.

7. Security Testing

- Validates data protection and system integrity.
- Includes testing for unauthorized access, authentication bypass, and data exposure risks.

CHAPTER 7

SYSTEM DEPLOYMENT

7.1 Operational Environment Specification

An effective operational environment ensures the reliability, availability, and performance of the Alumni Management System after deployment. The operational environment consists of appropriate hardware, software, and network configurations.

1. Hardware Specifications

Server Environment

- Server Type: Cloud-based or Dedicated Server
- Cloud Platforms: AWS (Amazon Web Services), Google Cloud Platform, or Vercel
- Processor: Minimum 4 vCPUs
- RAM: Minimum 8 GB (scalable based on user load)
- Storage: Minimum 100 GB SSD for fast data access and media storage
- Backup & Recovery: Scheduled automated backups and disaster recovery mechanisms

Client Environment

- Access through modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari
- Supported devices include desktops, laptops, tablets, and smartphones with internet connectivity

2. Software Specifications

- Frontend: React.js supporting modern web standards
- Backend: Node.js with Express.js for RESTful API development
- Database: MySQL / PostgreSQL using Sequelize ORM
- Authentication: JWT (JSON Web Tokens) for secure access control

Development Tools

- Visual Studio Code for development
- Postman for API testing
- Git and GitHub for version control and collaboration

3. Network Specifications

- Bandwidth: Minimum 100 Mbps for smooth data transfer
- Security Measures:
    - SSL/TLS certificates for encrypted communication
    - Firewalls and security groups to control network access
    - Network segmentation to protect sensitive services

4. Deployment Stages

- Development Stage: Feature implementation and unit testing
- Testing Stage: Functional, integration, and performance testing
- Staging Environment: Pre-production testing environment identical to production
- Production Environment: Live system accessible to alumni and administrators

5. Continuous Integration / Continuous Deployment (CI/CD)

- Automated CI/CD pipelines for testing and deployment
- Tools such as GitHub Actions, GitLab CI, or Jenkins for seamless updates

7.2 Conflicts and Challenges

Deploying and maintaining the Alumni Management System involves several technical, security, and operational challenges.

1. Technical Challenges

Compatibility Issues

- Ensuring seamless integration between frontend, backend, and database layers
- Maintaining browser compatibility for consistent user experience

Scalability Concerns

- Handling increasing numbers of alumni users
- Implementing load balancing and database optimization

Data Migration

- Migrating alumni data from legacy or manual systems
- Ensuring data accuracy and consistency during migration

Downtime Minimization

- Deploying updates during low-traffic periods
- Using rolling or blue-green deployment strategies

2. Security Challenges

Data Protection

- Securing sensitive alumni information using encryption
- Conducting regular security audits

Authentication and Authorization

- Secure token management and session handling
- Optional multi-factor authentication for administrators

Cyber Threat Prevention

- Protection against common web attacks such as XSS, CSRF, and DDoS
- Timely security patches and updates

3. Operational Challenges

Resource Allocation

- Balancing infrastructure costs with performance requirements
- Scaling resources during peak activity such as major alumni events

Monitoring and Maintenance

- Real-time monitoring using tools like New Relic or Prometheus
- Scheduled maintenance for updates and performance tuning

User Training and Support

- Training administrators to manage portal features
- Providing technical support for alumni users

4. Post-Deployment Challenges

User Feedback Management

- Collecting and analyzing feedback from alumni
- Implementing improvements based on feedback

Performance Optimization

- Continuous monitoring to improve load times and responsiveness
- Optimizing database queries and API performance

Bug Tracking and Updates

- Maintaining a bug-tracking system
- Regular release cycles for fixes and feature enhancements

CHAPTER 8

SYSTEM MAINTENANCE

8.1 Maintenance Procedures

Regular and structured maintenance ensures the Alumni Management System operates efficiently, remains secure, and continues to meet user expectations. Maintenance activities are categorized into preventive, corrective, adaptive, and perfective strategies.

1. Preventive Maintenance

Preventive maintenance aims to reduce the likelihood of system failures through regular monitoring and updates.

- **Software Updates:**
  - Regularly update Node.js, React.js, and database drivers to improve security and performance.
  - Apply security patches and bug fixes promptly.
- **Security Audits:**
  - Conduct periodic vulnerability assessments.
  - Review and update authentication mechanisms, especially JWT configurations.
  - Ensure secure password storage and access control policies.
- **Database Maintenance:**
  - Optimize database tables and indexes for frequently accessed alumni data.
  - Archive inactive alumni records and old logs to improve performance.
  - Schedule automated backups to ensure data recovery.
- **Infrastructure Maintenance:**
  - Monitor server resource usage (CPU, RAM, storage).
  - Perform load testing during peak alumni activities.
  - Scale cloud resources based on usage trends.
- **User Management:**
  - Review user roles and access privileges regularly.
  - Remove inactive or unauthorized user accounts to strengthen security.

2. Corrective Maintenance

Corrective maintenance focuses on resolving issues detected during system operation.

- **Bug Fixing:**
  - Log reported issues using a centralized issue-tracking system.
  - Prioritize and resolve critical bugs quickly.
- **Performance Optimization:**
  - Analyze system logs to identify slow API responses.
  - Optimize database queries and backend logic.
  - Improve frontend performance using lazy loading and caching.
- **Error Handling:**
  - Enhance error messages for better debugging.
  - Maintain structured error logs for faster issue resolution.

3. Adaptive Maintenance

Adaptive maintenance ensures compatibility with evolving technologies and institutional requirements.

- **Technology Upgrades:**
  - Upgrade frameworks, libraries, and dependencies as required.
  - Migrate to improved infrastructure or database solutions if needed.
- **Feature Enhancements:**
  - Collect feedback from alumni and administrators.
  - Introduce new features such as advanced networking tools or event enhancements.
- **Regulatory Compliance:**
  - Ensure compliance with data protection regulations.
  - Update privacy policies and terms of use accordingly.

4. Perfective Maintenance

Perfective maintenance improves system functionality and user satisfaction.

- **UI/UX Enhancements:**
  - Update user interface design for better usability and accessibility.

    - Improve navigation and responsiveness across devices.
- **Performance Tuning:**
    - Optimize server configurations and caching strategies.
    - Refine system workflows for faster processing.
- **Documentation Updates:**
    - Keep technical and user documentation up to date.
    - Document new features and maintenance procedures.

8.2 System Troubleshooting

System troubleshooting involves identifying, diagnosing, and resolving operational issues effectively using logs, tools, and user feedback.

1. Common System Issues and Solutions

- **Login Issues:**
    - Causes: Invalid credentials, expired JWT tokens, or server downtime.
    - Resolution: Validate credentials, refresh tokens, and restart authentication services if needed.
- **Database Connectivity Errors:**
    - Causes: Network issues, incorrect configuration, or database server downtime.
    - Resolution**:** Verify connection settings, check database service status, and restart services.
- **Slow System Performance:**
    - Causes: High user traffic, unoptimized queries, or heavy media content.
    - Resolution: Scale server resources, optimize database queries, and use caching mechanisms.
- **Job Posting or Event Creation Failures:**
    - Causes: API errors, incomplete form validation, or permission issues.
    - Resolution: Validate input data, review API logs, and ensure correct user permissions.

2. Diagnostic Tools and Methods

- **Logging Tools:**
  - Use backend logging tools such as Winston or Morgan.
  - Configure appropriate log levels for effective debugging.
- **Performance Monitoring:**
  - Use monitoring tools like Prometheus, Grafana, or New Relic.
  - Set alerts for performance anomalies.
- **Network Troubleshooting:**
  - Use tools like Ping and Traceroute to diagnose connectivity issues.
  - Verify firewall and security group configurations.
- **Database Troubleshooting:**
  - Monitor database performance and query execution times.
  - Perform regular health checks and indexing.

3. Troubleshooting Process

1. **Issue Identification:**
   - Collect details from error messages, logs, and user feedback.
2. **Diagnosis:**
   - Reproduce the issue in a test or staging environment.
   - Analyze logs and trace the root cause.
3. **Resolution:**
   - Apply fixes in the development or staging environment.
   - Perform testing before production deployment.
4. **Validation:**
   - Verify that the issue is resolved without impacting other modules.
5. **Documentation:**
   - Record the issue, cause, and solution for future reference.
6. **Preventive Action:**
   - Implement measures to avoid recurrence of similar issues.

CHAPTER 9

SYSTEM ENHANCEMENT

9.1 Future Extension

Continuous advancements in technology and evolving alumni expectations require periodic enhancements to the Alumni Management System to ensure long-term relevance, scalability, and improved user experience.

1. Scalability Enhancements

- **Multi-Institution Support:**
  - Extend the system to support multiple colleges or institutions using a single platform.
  - Implement institution-based data segregation for improved privacy and security.
- **Horizontal and Vertical Scaling:**
  - Enable vertical scaling by upgrading server resources.
  - Support horizontal scaling by adding multiple servers with load balancers for high availability.
  - Optimize cloud infrastructure to handle growing alumni participation.

2. Feature Expansion

- **AI-Based Alumni Insights:**
  - Use AI algorithms to analyze alumni engagement and activity trends.
  - Provide personalized event and job recommendations for alumni.
- **Mentorship & Collaboration Platform:**
  - Introduce mentor–mentee matching between senior alumni and students.
  - Enable discussion forums and professional collaboration spaces.
- **Mobile Application Development:**
  - Develop dedicated mobile applications for Android and iOS.
  - Provide push notifications and offline access to selected content.

- **Advanced Analytics & Reporting:**
  - Implement analytics dashboards to track engagement, event participation, and job portal usage.
  - Enable predictive analytics for planning future alumni initiatives.

3. Integration with External Systems

- **Social Media Integration:**
  - Integrate with platforms like LinkedIn for profile verification and networking.
  - Enable social sharing of alumni achievements and events.
- **Third-Party API Integration:**
  - Integrate email, SMS, and notification services.
  - Support payment gateways for event registrations or donations.
- **Digital Certificate & Badge Integration:**
  - Generate digital participation certificates for alumni events.
  - Issue digital badges for alumni contributions and achievements.

4. Security Enhancements

- **Advanced Authentication:**
  - Implement multi-factor authentication (MFA) for administrators.
  - Add optional biometric authentication in future mobile versions.
- **Data Protection:**
  - Strengthen data encryption for stored and transmitted information.
  - Implement audit logs for tracking administrative activities.
- **Enhanced Access Control:**
  - Introduce fine-grained role-based access control for advanced permissions.

5. User Experience Improvements

- **Customizable Dashboards:**
  - Allow alumni and administrators to personalize dashboards.

- **Multilingual Support:**
  - Add language options to support a diverse alumni base.
- **Accessibility Enhancements:**
  - Improve accessibility for differently-abled users following web accessibility standards.

9.2 Code Refactoring

Code refactoring is essential to maintain the performance, scalability, and maintainability of the Alumni Management System while preserving existing functionality.

1. Importance of Code Refactoring

- **Improved Readability:**
  - Simplifies complex logic for easier understanding and maintenance.
- **Enhanced Performance:**
  - Optimizes backend processes and frontend rendering.
- **Reduced Errors:**
  - Helps identify hidden bugs and potential issues early.
- **Scalability:**
  - Prepares the system for future feature enhancements.

2. Refactoring Strategies

- **Modular Architecture:**
  - Split large components into smaller, reusable modules.
  - Apply the Single Responsibility Principle (SRP).
- **Code Optimization:**
  - Remove redundant code.
  - Improve algorithms and database queries.
- **Consistent Naming Conventions:**
  - Use meaningful and standardized names for variables and functions.

- **Error Handling:**
  - Implement centralized error-handling mechanisms.
- **Documentation Updates:**
  - Keep technical and user documentation up to date.

3. Tools for Refactoring

- **Static Code Analysis:**
  - ESLint for identifying coding issues.
- **IDE Tools:**
  - Visual Studio Code refactoring features.
- **Version Control:**
  - Git for tracking changes and rollback.
- **Testing Frameworks:**
  - Jest for validating functionality after refactoring.

4. Refactoring Process

1. Code Review: Identify areas requiring refactoring.
2. Planning: Prioritize critical modules.
3. Implementation: Refactor incrementally.
4. Testing: Perform unit and integration testing.
5. Documentation: Update relevant documents.
6. Deployment: Validate changes in staging before production release.

5. Best Practices for Code Refactoring

- Perform refactoring regularly.
- Keep changes simple and purposeful.
- Avoid unnecessary optimizations.
- Use feature flags for gradual rollout of changes.

CHAPTER 10

CONCLUSION

The Alumni Management System represents a comprehensive and modern approach to managing alumni relations by transforming traditional, fragmented systems into a unified digital platform. Developed using contemporary web technologies, the system provides a secure, scalable, and user-friendly environment that strengthens the relationship between alumni and the institution. By centralizing alumni data, communication channels, job opportunities, events, and media content, the portal significantly improves operational efficiency and alumni engagement.

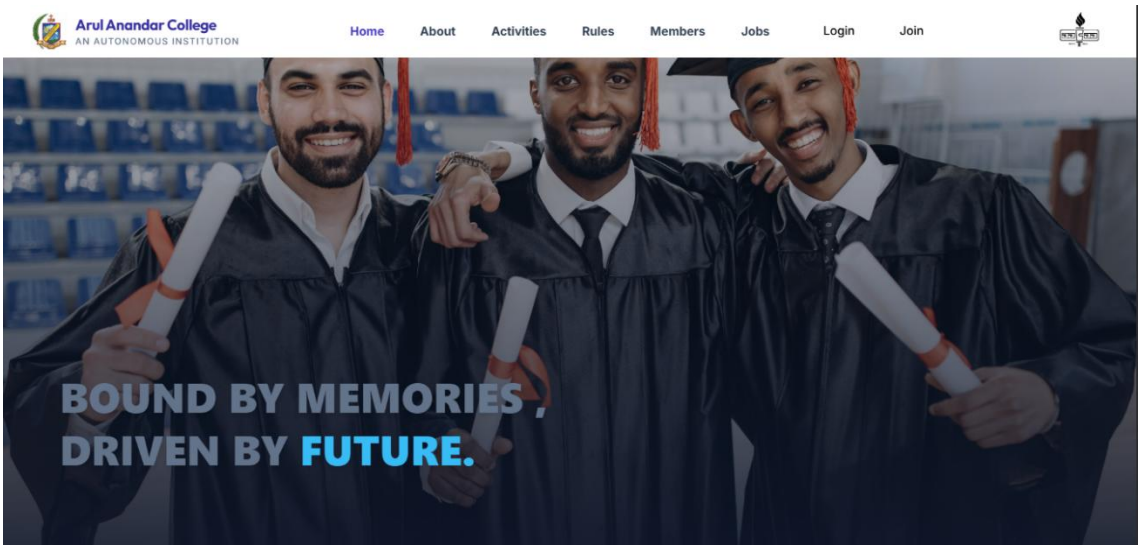
The platform enables alumni to actively participate in institutional activities, expand professional networks, and contribute to the growth of the college community. Features such as role-based authentication, structured alumni directories, job and career support modules, event and chapter management, and real-time notifications ensure effective interaction and transparency. The secure administrative dashboard empowers administrators to manage content, users, and system operations efficiently while maintaining data integrity and privacy.

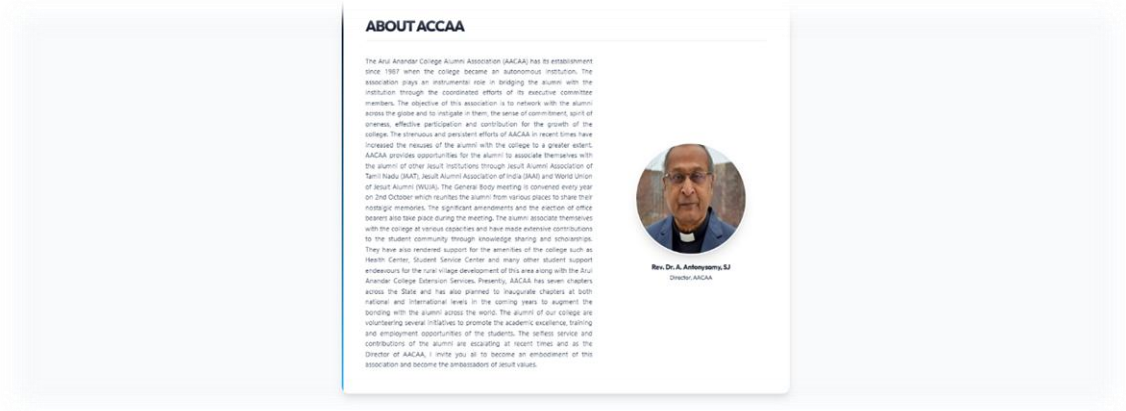
Furthermore, the system is designed with scalability and adaptability in mind, allowing it to accommodate a growing alumni base and evolving functional requirements. Continuous improvement strategies, including periodic maintenance, performance optimization, and code refactoring, ensure long-term reliability and system sustainability. Future enhancements such as mobile applications, advanced analytics, and intelligent networking features further extend the portal's capabilities.

In conclusion, the successful implementation of the Alumni Management System provides a robust digital foundation for sustained alumni engagement, institutional development, and professional collaboration. By fostering lifelong connections and leveraging technology-driven solutions, the system contributes to a stronger, more connected alumni community and supports the institution's vision for long-term growth and excellence.

APPENDIX – A & APPENDIX – B

Source Code and Screen Shots





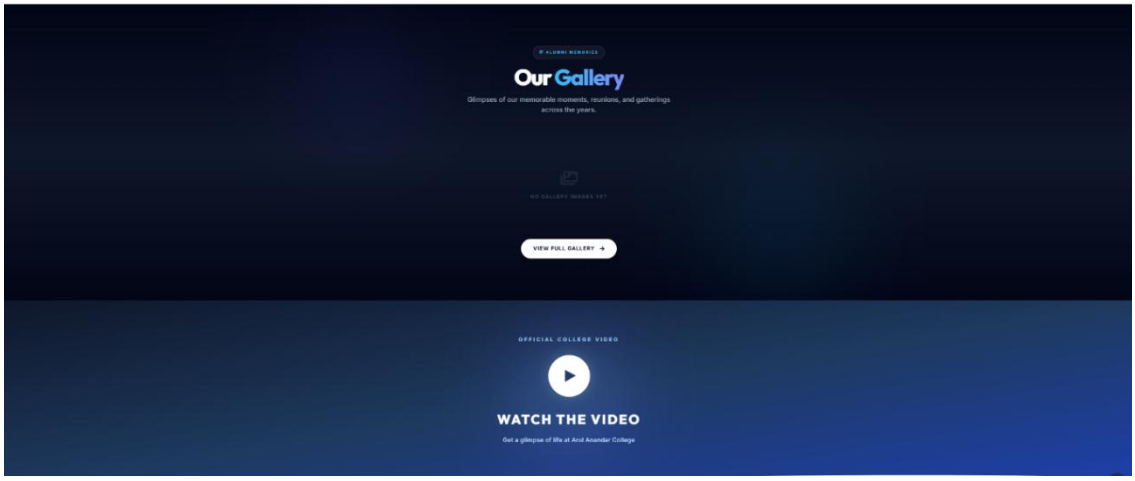


Figure 1: Home Page of Alumni Management System

Home.js:

```
// Triggers recompile
import React, { useState, useEffect } from "react";
import axios from "axios";
import { API_URL } from "../config";
import { Link, useLocation, useNavigate } from "react-router-dom";
import { MdOutlineMenu, MdClose } from "react-icons/md";
import rightLogo from "../assets/aacaalogo.png";

const Navbar = ({ isAdminPanel, hideAuthButtons, hideAdminLink,
isLoggedIn: propIsLoggedIn, onLogout: propOnLogout, isStatic,
disableAdminUI }) => {
  const [menuOpen, setMenuOpen] = useState(false);
  const [isSticky, setIsSticky] = useState(false);
  const location = useLocation();
  const navigate = useNavigate();

  const isLoggedIn = propIsLoggedIn !== undefined ? propIsLoggedIn :
!!sessionStorage.getItem("userEmail");
  const isAdminSession = !disableAdminUI &&
sessionStorage.getItem("isLoggedIn") === "true";

  const handleLogout = async () => {
    if (propOnLogout) {
      propOnLogout();
    } else {
      if (window.confirm("Logout?")) {
        const userEmail = sessionStorage.getItem("userEmail");
        if (userEmail) {
```

```
        try {
          await axios.post(`${API_URL}/api/auth/logout`, { email:
userEmail });
        } catch (err) {
          console.error("Logout error tracking:", err);
        }
      }
      sessionStorage.removeItem("userEmail");
      sessionStorage.removeItem("token");
      navigate("/");
    }
  }
};

useEffect(() => {
  const handleScroll = () => { // Scroll event
    setIsSticky(window.scrollY > 20);
  };
  window.addEventListener("scroll", handleScroll);
  return () => window.removeEventListener("scroll", handleScroll);
}, []);

useEffect(() => {
  setMenuOpen(false);
}, [location]);

// Lock body scroll when mobile menu is open
useEffect(() => {
```

```
  if (menuOpen) {
    document.body.style.overflow = 'hidden';
  } else {
    document.body.style.overflow = 'unset';
  }
}, [menuOpen]);

// Polling to check if user account still exists
useEffect(() => {
  const checkUserStatus = async () => {
    const userEmail = sessionStorage.getItem("userEmail");
    const isAdmin = sessionStorage.getItem("isLoggedIn") === "true";

    if (!userEmail || isAdmin) return;

    try {
      await axios.get(`${API_URL}/api/auth/profile/${userEmail}`);
    } catch (err) {
      if (err.response && err.response.status === 404) {
        sessionStorage.removeItem("userEmail");
        sessionStorage.removeItem("token");
        alert("Your account has been removed. Logging out.");
        navigate("/");
        window.location.reload();
      }
    }
  };
```

```
    const interval = setInterval(checkUserStatus, 5000);
    return () => clearInterval(interval);
  }, [navigate]);
  return (
    <nav className={`fixed top-0 left-0 w-full h-[70px] z-[1000] transition-all duration-300 flex items-center
      ${isSticky && !isStatic ? "bg-white shadow-md" : "bg-white"}
      ${isStatic ? "absolute" : ""}`}
    >
      {/* Mobile Menu Dropdown — outside the grid so it doesn't affect column layout */}
      <div className={`fixed top-[70px] left-0 right-0 bottom-0 bg-white/95 backdrop-blur-sm shadow-xl z-[4900] flex flex-col items-center gap-6 py-8 transition-all duration-300 md:hidden overflow-y-auto
        ${menuOpen ? "opacity-100 visible translate-y-0" : "opacity-0 invisible -translate-y-5 pointer-events-none"}`}
      >
        <Link to="/" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>Home</Link>
        <Link to="/about" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/about' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>About</Link>
        <Link to="/activities" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/activities' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>Activities</Link>
        <Link to="/rules" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/rules' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>Rules</Link>
```

```
<Link to="/members" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/members' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>Members</Link>

<Link to="/jobs" className={`text-[1.1rem] font-bold transition-colors ${location.pathname === '/jobs' ? 'text-indigo-700' : 'text-slate-800 hover:text-indigo-600'}`}>Jobs</Link>

{!isAdminPanel && !isAdminSession && (

<>

{!hideAuthButtons && (

<div className="flex flex-col w-full gap-4 px-12 mt-4">

{isLoggedIn ? (

<>

<Link to="/dashboard" className="btn w-full py-3 text-sm font-bold rounded-xl bg-slate-100 text-slate-900 hover:bg-slate-200 transition-all text-center">Profile Dashboard</Link>

<button className="btn w-full py-3 text-sm font-bold rounded-xl bg-indigo-600 text-white hover:bg-indigo-700 shadow-lg transition-all" onClick={handleLogout}>Sign Out</button>

</>

) : (

<>

<Link to="/login" className="btn w-full py-3 text-sm font-bold rounded-xl bg-transparent border-2 border-slate-900 text-slate-900 hover:bg-slate-900 hover:text-white transition-all text-center">Login</Link>

<Link to="/signup" className="btn w-full py-3 text-sm font-bold rounded-xl bg-transparent border-2 border-slate-900 text-slate-900 hover:bg-slate-900 hover:text-white transition-all text-center">Join Community</Link>

</>

)}

</div>
```

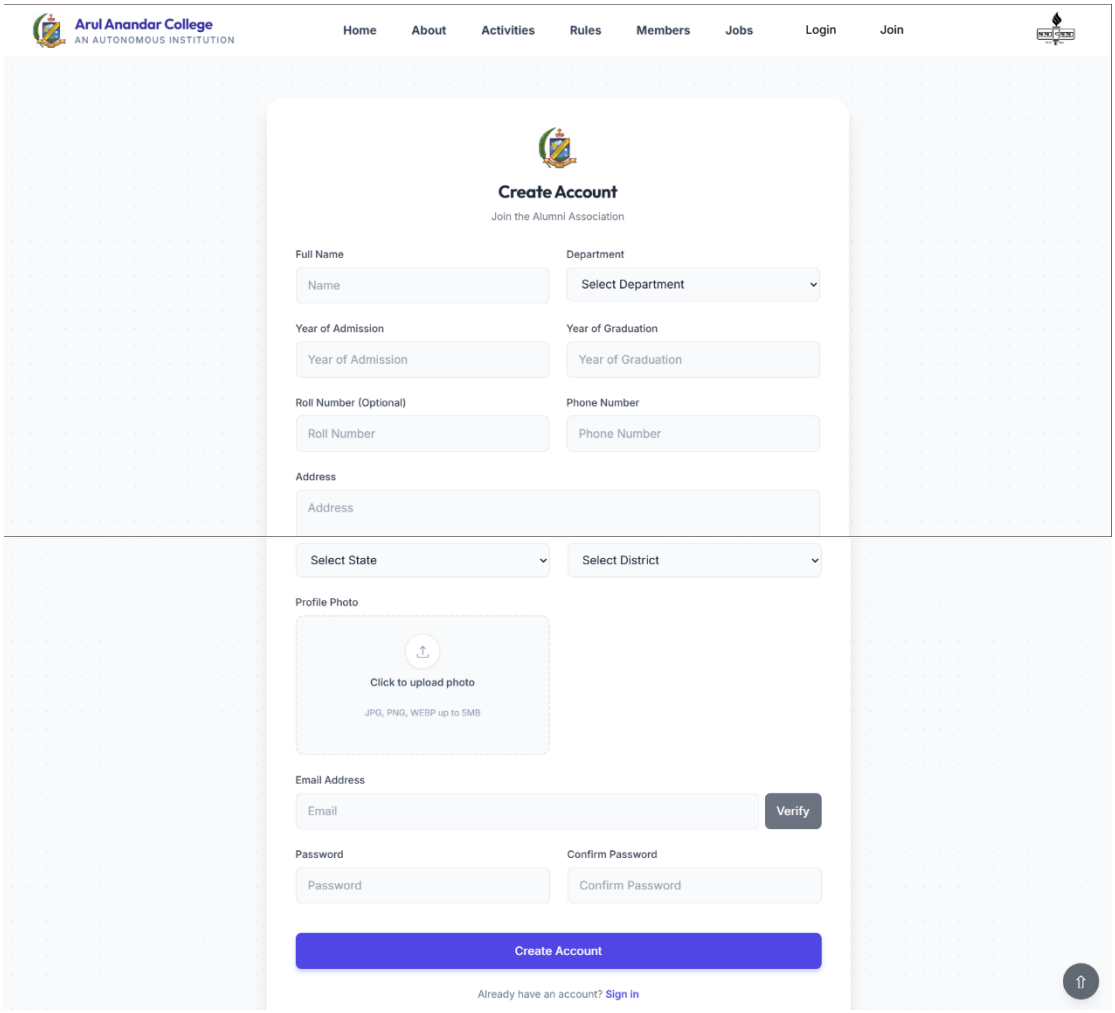
```
          )}
        </>
      )}
      {isAdminSession && (
        <div className="flex flex-col w-full gap-4 px-12 mt-4">
          <Link to="/AdminPage" className="btn w-full py-3 text-sm font-bold rounded-xl bg-amber-100 text-amber-800 hover:bg-amber-200 transition-all text-center">Admin Dashboard</Link>
          <button className="btn w-full py-3 text-sm font-bold rounded-xl bg-red-50 text-red-600 hover:bg-red-100 shadow-sm transition-all" onClick={() => {
            if (window.confirm("Logout from Admin Panel?")) {
              sessionStorage.removeItem("isLoggedIn");
              navigate("/");
            }
          }}>Admin Sign Out</button>
        </div>
      )}
    </div>
    {/* 3-column grid: exactly 3 children — left logo | center nav | right logo+hamburger */}
    <div className="w-full h-full grid grid-cols-[auto_1fr_auto] items-center px-2 md:px-4">
      {/* CHILD 1 — LEFT: College Logo */}
      <Link to="/" className={`flex items-center gap-2 md:gap-3 no-underline flex-shrink-0 pl-3 md:pl-6 ${menuOpen ? 'invisible' : ''}`}>
        <img src={process.env.PUBLIC_URL + "/college_logo.jpg"} alt="AAC Logo" className="h-10 md:h-12 w-auto mix-blend-multiply flex-shrink-0" />
        <div className="flex flex-col">
```

```
          <h2 className="text-sm md:text-xl font-bold text-indigo-800 leading-tight m-0 whitespace-nowrap">Arul Anandar College</h2>

          <p className="text-[10px] md:text-xs font-medium text-slate-500 uppercase tracking-widest m-0 whitespace-nowrap">An AUTONOMOUS Institution</p>

        </div>

      </Link>

      {/* CHILD 2 — CENTER: Desktop Nav Links */}

      <div className="hidden md:flex flex-row items-center justify-center gap-8 lg:gap-12">

        <Link to="/" className={`text-base font-bold transition-colors ${location.pathname === '/' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>Home</Link>

        <Link to="/about" className={`text-base font-bold transition-colors ${location.pathname === '/about' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>About</Link>

        <Link to="/activities" className={`text-base font-bold transition-colors ${location.pathname === '/activities' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>Activities</Link>

        <Link to="/rules" className={`text-base font-bold transition-colors ${location.pathname === '/rules' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>Rules</Link>

        <Link to="/members" className={`text-base font-bold transition-colors ${location.pathname === '/members' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>Members</Link>

        <Link to="/jobs" className={`text-base font-bold transition-colors ${location.pathname === '/jobs' ? 'text-indigo-700' : 'text-slate-700 hover:text-indigo-600'}`}>Jobs</Link>

        {!isAdminPanel && !isAdminSession && (

          <>

            {!hideAuthButtons && (

              <div className="flex items-center gap-3">
```

```
{isLoggedIn ? (
  <>
    <Link to="/dashboard" className="btn px-5 py-2 text-sm font-bold rounded-full bg-transparent border-2 border-black text-black hover:bg-indigo-600 hover:text-white hover:border-indigo-600 transition-all text-center">Profile</Link>
    <button className="btn px-5 py-2 text-sm font-bold rounded-full bg-white text-black border-2 border-black hover:bg-indigo-600 hover:text-white hover:border-indigo-600 shadow-sm hover:shadow-md transition-all" onClick={handleLogout}>Sign Out</button>
  </>
) : (
  <>
    <Link to="/login" className="btn px-5 py-2 text-sm font-bold rounded-full bg-transparent border-2 border-black text-black hover:bg-indigo-600 hover:text-white hover:border-indigo-600 transition-all text-center">Login</Link>
    <Link to="/signup" className="btn px-5 py-2 text-sm font-bold rounded-full bg-transparent border-2 border-black text-black hover:bg-indigo-600 hover:text-white hover:border-indigo-600 transition-all text-center">Join</Link>
  </>
)}
</div>
)}
</>
)}
{isAdminSession && (
<div className="flex items-center gap-3">
<Link to="/AdminPage" className="btn px-5 py-2 text-sm font-bold rounded-full bg-transparent border-2 border-black text-black
```

```
hover:bg-indigo-600 hover:text-white hover:border-indigo-600 transition-all text-center">Admin Profile</Link>
            <button className="btn px-5 py-2 text-sm font-bold rounded-full bg-white text-black border-2 border-black hover:bg-indigo-600 hover:text-white hover:border-indigo-600 shadow-sm hover:shadow-md transition-all" onClick={() => {
              if (window.confirm("Logout from Admin Panel?")) {
                sessionStorage.removeItem("isLoggedIn");
                navigate("/");
              }
            }}>Sign Out</button>
          </div>
        )}
      </div>
      {/* CHILD 3 — RIGHT: Alumni Logo + Mobile Hamburger */}
      <div className="flex items-center justify-end gap-2 flex-shrink-0 pr-3 md:pr-6">
        <img src={rightLogo} alt="AAC Alumni" className="hidden md:block h-[60px] w-auto mix-blend-multiply flex-shrink-0" />
        <button className="text-2xl text-slate-800 md:hidden" onClick={() => setMenuOpen(!menuOpen)}>
          {menuOpen ? <MdClose /> : <MdOutlineMenu />}
        </button>
      </div>


    </div>
  </nav>
);
};
export default Navbar;
```

Login.js:



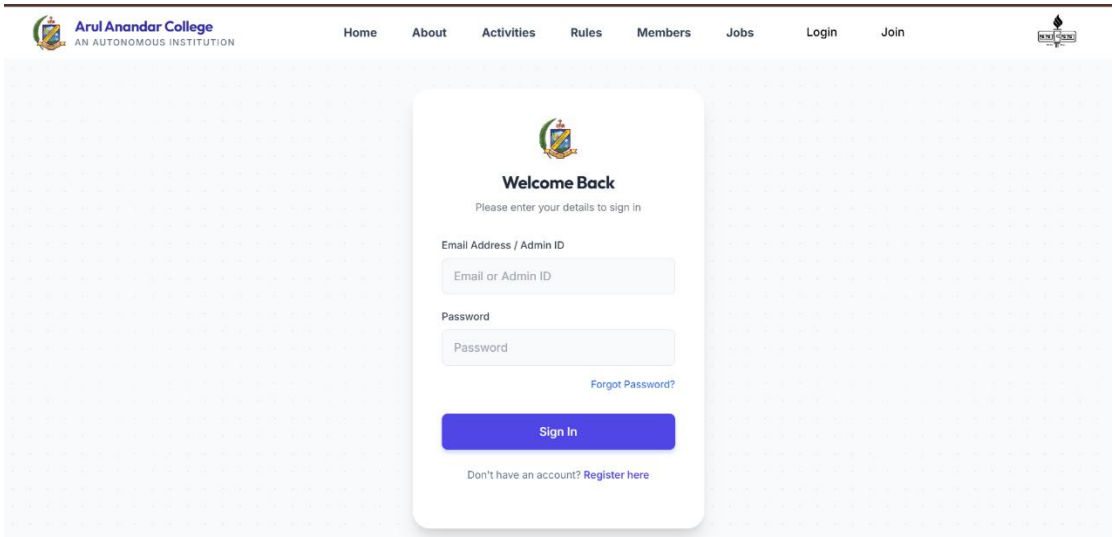


Figure 2: Login and Join Page of Alumni Management System

```
import React, { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import axios from "axios";
import { API_URL } from "../config";
import Navbar from "./Navbar";
const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
  const handleLogin = async (e) => {
    e.preventDefault();
    setLoading(true);
    // Check for Admin Credentials
    if (email === "aacadmin" && password === "alumniadmin") {
      sessionStorage.setItem("isLoggedIn", "true");
      setLoading(false);
      navigate("/AdminPage");
      return;
    }
    try {
      const response = await axios.post(`${API_URL}/api/auth/login`, { email, password });
      sessionStorage.setItem("token", response.data.token); // Save token
      sessionStorage.setItem("userEmail", email); // Store the email
      // sessionStorage.removeItem("isLoggedIn"); // In a separate tab, this won't affect admin
session
      setLoading(false);
      navigate("/dashboard"); // Redirect to User Dashboard
    } catch (err) {
      setError(err.response?.data?.message || "Login failed. Please try again.");
      setLoading(false);
    }
```

```
  };
  return (
    <>
      <Navbar />
      <div className="min-h-screen flex items-center justify-center p-8 bg-slate-50 bg-[radial-gradient(#e2e8f0_1px,transparent_1px)] bg-[size:24px_24px]">
        <div className="w-full max-w-[400px] bg-white rounded-3xl p-10 shadow-xl mt-20 fade-in">
          <div className="text-center mb-8">
            <img src={process.env.PUBLIC_URL + "/college_logo.jpg"} alt="Logo" className="w-14 h-14 object-contain mx-auto mb-4 mix-blend-multiply" loading="lazy" />
            <h2 className="text-2xl font-bold mb-2 text-gray-800">Welcome Back</h2>
            <p className="text-sm text-gray-500 m-0">Please enter your details to sign in</p>
          </div>
          <form onSubmit={handleLogin} className="flex flex-col gap-5">
            <div className="flex flex-col gap-2">
              <label htmlFor="email" className="text-sm font-medium text-gray-700">Email Address / Admin ID</label>
              <input
                type="text"
                id="email"
                placeholder="Email or Admin ID"
                value={email}
                autoComplete="username"
                onChange={(e) => setEmail(e.target.value)}
                className="w-full px-4 py-3 border border-gray-200 rounded-lg text-base transition-all bg-slate-50 focus:outline-none focus:border-indigo-600 focus:bg-white focus:ring-4 focus:ring-indigo-600/10"
              />
            </div>
            <div className="flex flex-col gap-2">
              <label htmlFor="password" className="text-sm font-medium text-gray-700">Password</label>
              <input
                type="password"
```

```
            id="password"
            placeholder="Password"
            value={password}
            autoComplete="current-password"
            onChange={(e) => setPassword(e.target.value)}
            className="w-full px-4 py-3 border border-gray-200 rounded-lg text-base transition-all bg-slate-50 focus:outline-none focus:border-indigo-600 focus:bg-white focus:ring-4 focus:ring-indigo-600/10"
          />
          <div className="text-right mt-1">
            <Link to="/forgot-password" style={{ fontSize: "0.85rem", color: "#2563eb" }} className="hover:underline">Forgot Password?</Link>
          </div>
        </div>
        <button
          type="submit"
          className={`w-full mt-2 px-6 py-3 bg-indigo-600 hover:bg-indigo-700 text-white font-medium rounded-lg transition-colors shadow-md shadow-indigo-200 ${loading ? 'btn-loading' : ''}`}
        >
          Sign In
        </button>
        {error && <div className="text-red-500 text-sm text-center bg-red-50 p-2 rounded-lg border border-red-100">{error}</div>}
      </form>
      <div className="text-center mt-6 text-sm text-gray-500">
        <p>Don't have an account? <Link to="/signup" className="font-semibold text-indigo-600 hover:text-indigo-700">Register here</Link></p>
      </div>
    </div>
  </div>
</>
);
};
export default Login;
```

AdminPage.js

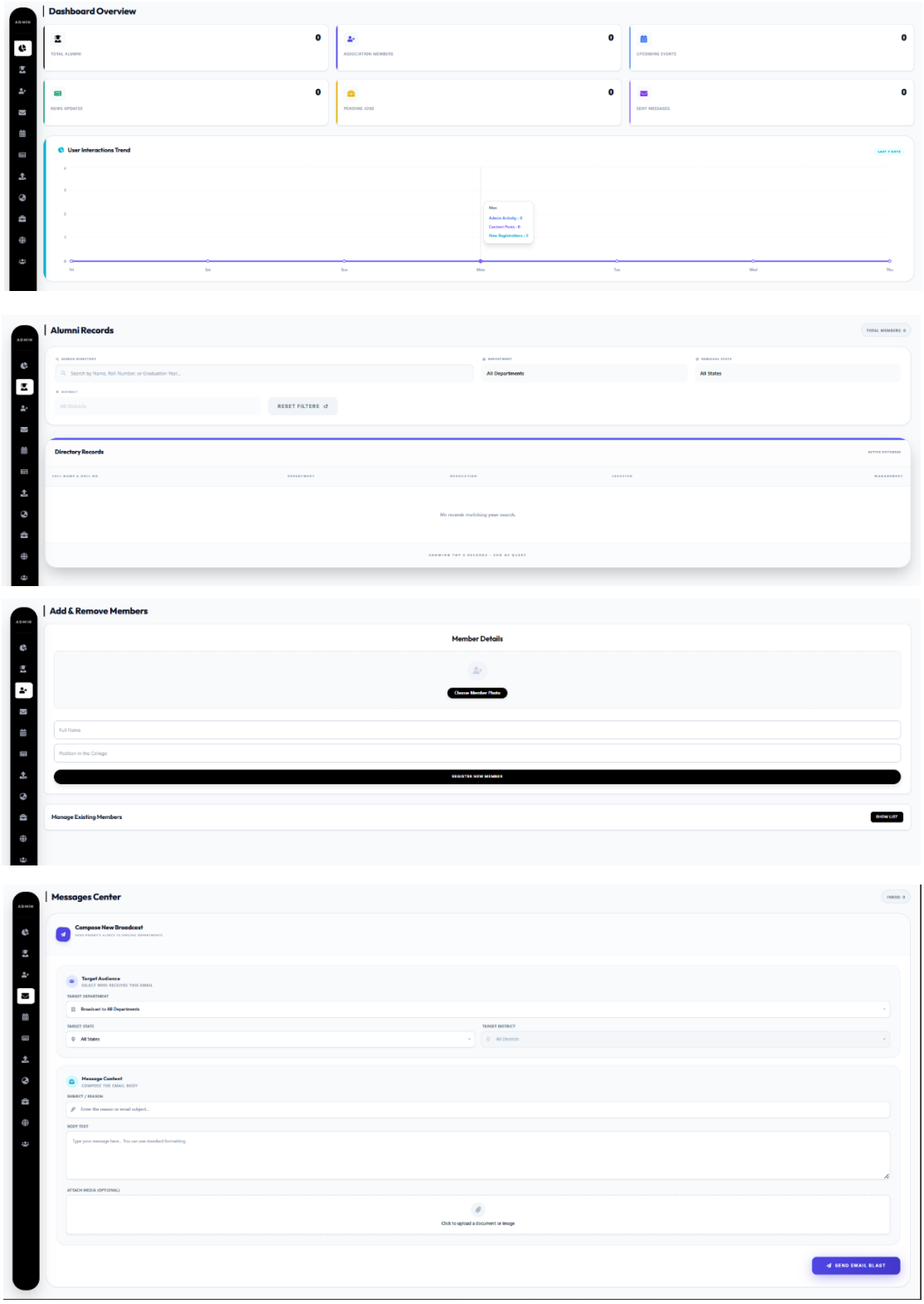


Figure 3: Admin Page of Alumni Management System

```
import React, { useState, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import { API_URL } from "../config";
import Navbar from "./Navbar";
import MessageSendPage from "../components/MessageSend";
import {
  FaEnvelope, FaUpload, FaChartPie, FaSearch, FaFilePdf, FaCalendarAlt, FaNewspaper,
  FaUserPlus, FaUserGraduate, FaMapMarkerAlt, FaKey, FaCloudUploadAlt, FaCheckCircle, FaTrash, FaGlobeAsia, FaBriefcase,
  FaCogs, FaPaperPlane, FaInbox, FaUsers, FaEdit, FaTimes, FaGlobe
} from "react-icons/fa";
import { defaultMembers } from "../data/membersData";
import { indianStates, districtsByState } from "../data/locationData";
import AdminBatchView from "./AdminBatchView";
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts';

const AdminPage = () => {
  const historyRef = useRef(null);
  const editingMessageFileRef = useRef(null);
  const [activeTab, setActiveTab] = useState("overview"); // Tab state
  const [users, setUsers] = useState([]);

  const [filteredUsers, setFilteredUsers] = useState([]);
  const [selectedAlumni, setSelectedAlumni] = useState(null);
  const [showAlumniModal, setShowAlumniModal] = useState(false);

  // Prevent background scroll when modal is open
  useEffect(() => {
```

```
  if (showAlumniModal) {
    document.body.style.overflow = "hidden";
  } else {
    document.body.style.overflow = "auto";
  }
  return () => { document.body.style.overflow = "auto"; };
}, [showAlumniModal]);

// Filters
const [selectedDepartment, setSelectedDepartment] = useState("");
const [searchQuery, setSearchQuery] = useState("");
const [selectedState, setSelectedState] = useState("");
const [selectedDistrict, setSelectedDistrict] = useState("");
const [availableDistricts, setAvailableDistricts] = useState([]);
const [eventAvailableDistricts, setEventAvailableDistricts] = useState([]);

useEffect(() => {
  if (selectedState) {
    const districts = districtsByState[selectedState] || [];
    setAvailableDistricts(districts.includes("Other") ? districts : [...districts, "Other"]);
    setSelectedDistrict("");
  } else {
    setAvailableDistricts([]);
    setSelectedDistrict("");
  }
}, [selectedState]);

// File Upload State
const [galleryFile, setGalleryFile] = useState(null);
const [pdfFile, setPdfFile] = useState(null);
```

```
  const [imageTitle, setImageTitle] = useState("");
  const [imageLocation, setImageLocation] = useState("");
  const [galleryImages, setGalleryImages] = useState([]);
  const [editingGalleryId, setEditingGalleryId] = useState(null);
  const [galleryEditForm, setGalleryEditForm] = useState({ title: "", location: "" });

  // PDF Upload Metadata
  const [pdfCategory, setPdfCategory] = useState("");
  const [chapterCity, setChapterCity] = useState("");

  // Event State
  const [events, setEvents] = useState([]);
  const [eventForm, setEventForm] = useState({
    title: "", date: "", type: "General", otherType: "", description: "", location: "",
    targetDepartment: "All", targetState: "All", targetDistrict: "All", allowAttendance:
true
  });
  const [selectedEventAttendees, setSelectedEventAttendees] = useState([]);
  const [showAttendeesModal, setShowAttendeesModal] = useState(false);
  const [isFetchingAttendees, setIsFetchingAttendees] = useState(false);
  const [attendeeCounts, setAttendeeCounts] = useState({}); // { eventId: count }

  const [news, setNews] = useState([]);
  const [newsForm, setNewsForm] = useState({
    title: "", content: "", category: "General", otherCategory: ""
  });

  useEffect(() => {
    if (eventForm.targetState && eventForm.targetState !== "All") {
      const districts = districtsByState[eventForm.targetState] || [];
```

```
    setEventAvailableDistricts(districts.includes("Other") ? districts : [...districts,
"Other"]);
  } else {
    setEventAvailableDistricts([]);
  }
}, [eventForm.targetState]);


// Messages State
const [inboxMessages, setInboxMessages] = useState([]);
const [sentMessages, setSentMessages] = useState([]);
const [editingMessage, setEditingMessage] = useState(null); // { id, content }
const [editingMessageFile, setEditingMessageFile] = useState(null);
const [viewingMessage, setViewingMessage] = useState(null); // For modal

const [editingNewsId, setEditingNewsId] = useState(null);
const [newsEditForm, setNewsEditForm] = useState({
  title: "", content: "", category: "", otherCategory: ""
});

// Add Member State
const [memberForm, setMemberForm] = useState({
  name: "", position: "", image: null
});
const [members, setMembers] = useState([]);
const [showMembers, setShowMembers] = useState(false);
const [editingId, setEditingId] = useState(null);
const [editForm, setEditForm] = useState({ name: "", position: "" });

const [uploadedFiles, setUploadedFiles] = useState([]);
const [historyType, setHistoryType] = useState("");
```

```
  const [editingFileId, setEditingFileId] = useState(null);
  const [fileEditForm, setFileEditForm] = useState({ originalName: "", category: "", chapterCity: "", file: null });
  const [isSavingFile, setIsSavingFile] = useState(false);

  // Chapter State
  const [chapters, setChapters] = useState([]);
  const [chapterForm, setChapterForm] = useState({ name: "", description: "", image: null });
  const [editingChapterId, setEditingChapterId] = useState(null);
  const [chapterEditForm, setChapterEditForm] = useState({ name: "", description: "", image: null });

  // Loading States
  const [isAddingMember, setIsAddingMember] = useState(false);
  const [isSavingMember, setIsSavingMember] = useState(false);
  const [isSavingEvent, setIsSavingEvent] = useState(false);
  const [isSavingNews, setIsSavingNews] = useState(false);
  const [isUpdatingMessage, setIsUpdatingMessage] = useState(false);
  const [isUploadingGallery, setIsUploadingGallery] = useState(false);
  const [isUploadingPdf, setIsUploadingPdf] = useState(false);
  const [isSavingChapter, setIsSavingChapter] = useState(false);
  const [isSavingJob, setIsSavingJob] = useState(false);
  const [isUpdatingJobStatus, setIsUpdatingJobStatus] = useState(false);
  const [siteContent, setSiteContent] = useState({ aboutText: "", aboutImage: "", videoUrl: "", videoFile: "", videoType: "link" });
  const [isSavingAbout, setIsSavingAbout] = useState(false);
  const [isSavingVideo, setIsSavingVideo] = useState(false);
  const [aboutImageFile, setAboutImageFile] = useState(null);
  const [videoUploadFile, setVideoUploadFile] = useState(null);

  // Jobs State
```

```
const [jobRequests, setJobRequests] = useState([]);
const [jobFilter, setJobFilter] = useState("Pending"); // Pending, Approved, Rejected

const getYouTubeEmbedUrl = (url) => {
  if (!url) return "";
  if (url.includes("embed/")) return url;
  let videoId = "";
  if (url.includes("v=")) {
    videoId = url.split("v=")[1].split("&")[0];
  } else if (url.includes("youtu.be/")) {
    videoId = url.split("youtu.be/")[1].split("?")[0];
  }
  return videoId ? `https://www.youtube.com/embed/${videoId}` : url;
};

const handleAddMember = async () => {
  if (!memberForm.name || !memberForm.position) {
    return alert("Name and Position are required");
  }

  setIsAddingMember(true);
  const newMember = {
    id: Date.now(),
    name: memberForm.name,
    position: memberForm.position,
    image: null
  };
```

UserPage.js:

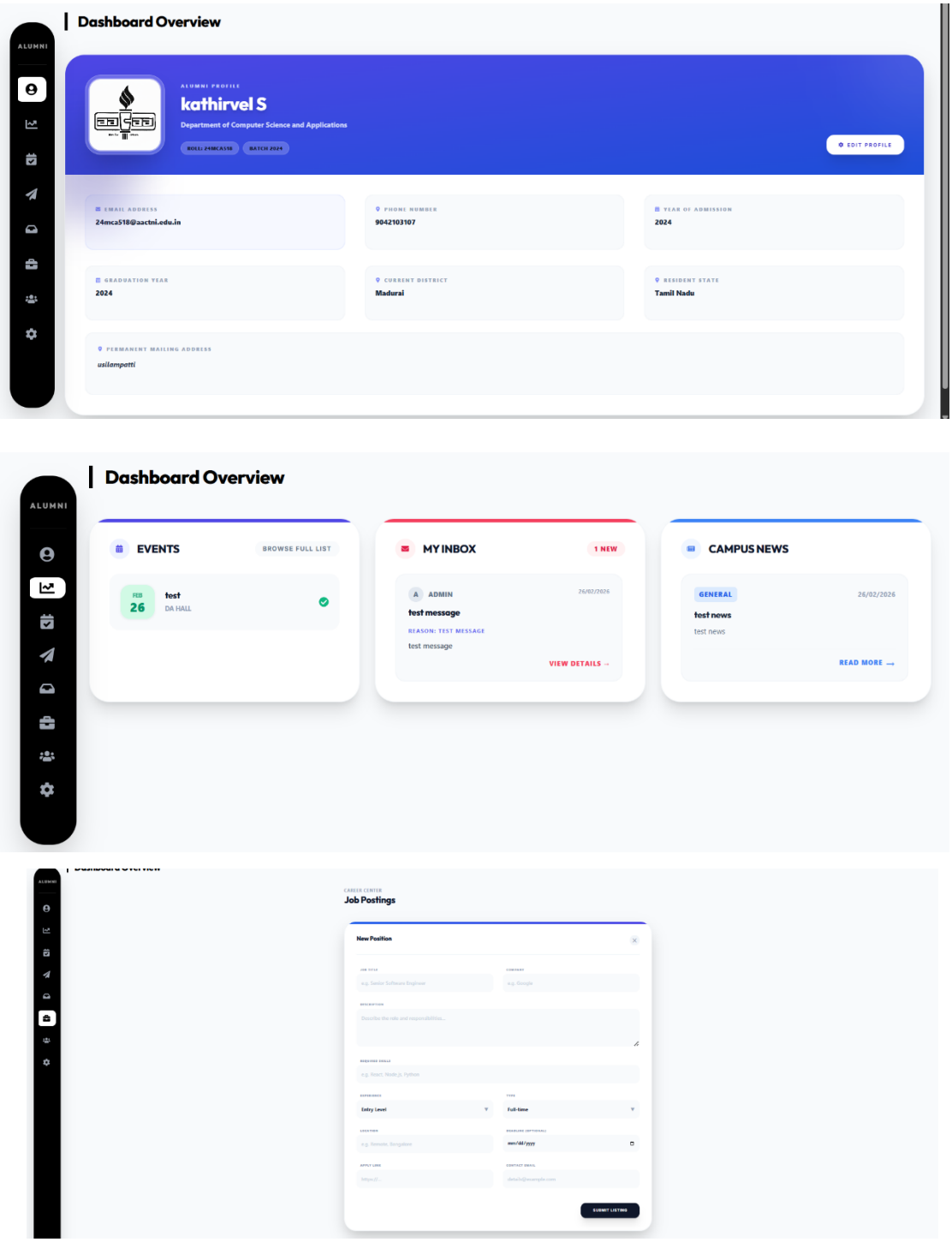


Figure 4: User Page of Alumni Management System

```
import React, { useState, useEffect, useCallback, useRef } from "react";
import axios from "axios";
import Navbar from "./Navbar";
import { API_URL } from "../config";

import {
  FaCalendarAlt,
  FaEnvelope,
  FaMapMarkerAlt,
  FaChartLine,
  FaCog,
  FaPaperPlane,
  FaInbox,
  FaUserCircle,
  FaTimes,
  FaBriefcase,
  FaUpload,
  FaCheckCircle,
  FaCalendarCheck,
  FaMapPin,
  FaTag,
  FaFilePdf,
  FaNewspaper,
  FaUsers,
  FaPaperclip
} from "react-icons/fa";
import BatchGroup from "./BatchGroup";
import { subscribeUserToPush } from "../pushNotifications";

const Dashboard = () => {
```

```
  const [userEmail] = useState(sessionStorage.getItem("userEmail") || "Guest");

  const [activeTab, setActiveTab] = useState("details");
  const [userProfile, setUserProfile] = useState({});
  const [isEditing, setIsEditing] = useState(false);
  const [editForm, setEditForm] = useState({});
  const [selectedImage, setSelectedImage] = useState(null);
  const [isUpdatingProfile, setIsUpdatingProfile] = useState(false);

  const fetchProfile = useCallback(async () => {
    try {
      const res = await axios.get(`${API_URL}/api/auth/profile/${userEmail}`);
      setUserProfile(res.data);
      setEditForm(res.data);
    } catch (err) {
      console.error(err);
    }
  }, [userEmail]);


  useEffect(() => {
    // sessionStorage.removeItem("isLoggedIn"); // Removed to avoid clearing across
roles
    if (userEmail !== "Guest") {
      fetchProfile();
      subscribeUserToPush(userEmail); // Register for Push Notifications
    }
  }, [userEmail, fetchProfile]);

  const handleUpdateProfile = async () => {
    setIsUpdatingProfile(true);
```

```
  try {
    const formData = new FormData();
    formData.append("email", userEmail);
    formData.append("name", editForm.name || "");
    formData.append("Phone", editForm.Phone || "");
    formData.append("Address", editForm.Address || "");
    formData.append("district", editForm.district || "");
    formData.append("state", editForm.state || "");
    formData.append("Department", editForm.Department || "");
    formData.append("year", editForm.year || "");
    formData.append("graduationYear", editForm.graduationYear || "");
    formData.append("roll", editForm.roll || "");

    if (selectedImage) {
      formData.append("profilePhoto", selectedImage);
    }

    await axios.put(`${API_URL}/api/auth/profile/update`, formData, {
      headers: { "Content-Type": "multipart/form-data" }
    });
    alert("Profile updated successfully!");
    setIsEditing(false);
    fetchProfile();
  } catch (err) {
    console.error(err);
    alert("Failed to update profile");
  } finally {
    setIsUpdatingProfile(false);
  }
};
```

```
  /* Messages State */
  const [messages, setMessages] = useState([]); // Inbox
  const [sentMessages, setSentMessages] = useState([]); // User Sent History
  const [userMessage, setUserMessage] = useState(""); // For sending to admin
  const [messageSubject, setMessageSubject] = useState(""); // Reason/Subject for
admin msg
  const [events, setEvents] = useState([]);
  const [news, setNews] = useState([]);
  const [editingMessage, setEditingMessage] = useState(null); // {id, content}
  const [editingMessageFile, setEditingMessageFile] = useState(null);
  const [userMessageFile, setUserMessageFile] = useState(null);
  const [isSendingToAdmin, setIsSendingToAdmin] = useState(false);
  const [isUpdatingMessage, setIsUpdatingMessage] = useState(false);
  const [userAttendingEvents, setUserAttendingEvents] = useState([]); // List of event
IDs user is attending
  const [isJoiningEvent, setIsJoiningEvent] = useState(null); // ID of event being
joined
  const [viewingMessage, setViewingMessage] = useState(null); // For modal


  /* Jobs State */
  const [userJobs, setUserJobs] = useState([]);
  const [viewingNews, setViewingNews] = useState(null); // For news modal
  const [showJobForm, setShowJobForm] = useState(false);
  const [jobForm, setJobForm] = useState({
    title: "", company: "", description: "", skills: "", experienceLevel: "Entry Level",
    location: "", applyLink: "", contactEmail: "", jobType: "Full-time", deadline: ""
  });
  const [editingJobId, setEditingJobId] = useState(null);
  const [isSavingJob, setIsSavingJob] = useState(false);
```

```
  // Refs for file inputs
  const userMessageFileRef = useRef(null);
  const editingMessageFileRef = useRef(null);

  const fetchMessages = useCallback(async () => {
    if (userEmail === "Guest") return;
    try {
      const dept = userProfile.Department ?
encodeURIComponent(userProfile.Department) : "";

      // Fetch Inbox
      const responseInbox = await
axios.get(`${API_URL}/api/messages/${userEmail}?department=${dept}`);
      setMessages(responseInbox.data);

      // Fetch Sent History
      const responseSent = await
axios.get(`${API_URL}/api/messages/${userEmail}?type=sent`);
      setSentMessages(responseSent.data);

    } catch (error) {
      console.error("Error fetching messages:", error);
    }
  }, [userEmail, userProfile.Department]);

  /* Notifications State */

  useEffect(() => {

    const fetchEvents = async () => {
      try {
```

```
      const response = await
axios.get(`${API_URL}/api/events?email=${userEmail}`);
      setEvents(response.data);

      // If user logged in, fetch which events they are attending
      if (userEmail !== "Guest") {
        const attendanceRes = await
axios.get(`${API_URL}/api/events/user/${userEmail}/attendance`);
        setUserAttendingEvents(attendanceRes.data);
      }
    } catch (error) {
      console.error("Error fetching events:", error);
    }
  };

  const fetchNews = async () => {
    try {
      const response = await
axios.get(`${API_URL}/api/news?email=${userEmail}`);
      setNews(response.data);
    } catch (error) {
      console.error("Error fetching news:", error);
    }
  };

  fetchEvents();
  fetchNews();

  if (userEmail !== "Guest") {
    fetchMessages();
  }
```

```
}, [userEmail, userProfile.Department, fetchMessages]);

const handleSendMessageToAdmin = async () => {
  if (!userMessage.trim()) return alert("Message cannot be empty");

  setIsSendingToAdmin(true);
  const formData = new FormData();
  formData.append("sender", userEmail);
  formData.append("recipientEmail", "Admin");
  formData.append("content", userMessage);
  formData.append("subject", messageSubject);

  if (userMessageFile) {
    formData.append("attachment", userMessageFile);
  }

  try {
    await axios.post(`${API_URL}/api/messages`, formData, {
      headers: { "Content-Type": "multipart/form-data" },
    });
```

BIBLIOGRAPHY

- Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
- Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education, 2015.
- Elmasri and Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016.
- Silberschatz, Korth, and Sudarshan, *Database System Concepts*, 6th Edition, McGraw-Hill Education, 2011.
- David Flanagan, *JavaScript: The Definitive Guide*, 7th Edition, O'Reilly Media, 2020.
- Kyle Simpson, *You Don't Know JS Yet*, 2nd Edition, O'Reilly Media, 2020.
- Adam Freeman, *Pro React 16*, Apress, 2019.
- Brad Dayley, *Node.js, MongoDB, and Angular Web Development*, Pearson Education, 2018.

WEB REFERENCES

1. React Documentation, *"React – A JavaScript library for building user interfaces"*,
   Available at: https://react.dev
2. Node.js Documentation, *"Node.js® JavaScript runtime"*,
   Available at: https://nodejs.org
3. Express.js Documentation, *"Fast, unopinionated, minimalist web framework for Node.js"*,
   Available at: https://expressjs.com
4. MySQL Documentation, *"MySQL Database Management System"*,
   Available at: https://dev.mysql.com/doc
5. Sequelize Documentation, *"Sequelize ORM for Node.js"*,
   Available at: https://sequelize.org
6. MDN Web Docs, *"JavaScript, HTML, and CSS Documentation"*,
   Available at: https://developer.mozilla.org
7. Tailwind CSS Documentation, *"Utility-first CSS framework"*,
   Available at: https://tailwindcss.com/docs
8. Multer Documentation, *"Node.js middleware for handling file uploads"*,
   Available at: https://www.npmjs.com/package/multer
9. W3Schools, *"Web Development Tutorials"*,
   Available at: https://www.w3schools.com